The built-in `Array::sort` can sort 150,000 random numbers in milliseconds, not seconds like your competitor sorting. The slow sorting is caused by the comparison function. Why? Because the `compareOnLastName` function is slow. When the array is sorted, every item might need to be compared to every other item! That's more comparisons than you want to see written down, quite apart from the effect it has on the performance of your program. To avoid this problem, your first thought is to optimize the sorting to be fast enough.

9.1.4. Decorate, sort, undecorate

It's always faster to do nothing than to do something. Consider what the *something* is that happens inside the `compareOnLastName` function:

```
compareOnLastName = (a,b) ->
  lastName = (s) ->
    s.split(/\s+/g)[1].replace /,/, ''
  if !a or !b
    1
  else if lastName(a) >= lastName(b)
    1
  else
    -1
```

Last name occurs after the first space.

Return 1 if either last name is missing.

Two last names are extracted for each comparison, which is expensive and unnecessary because the last name only needs to be determined *once* for each competitor. That can be done before any sorting occurs. A technique known as *decorate-sort-undecorate,* which uses this idea, is demonstrated here.

Listing 9.2. Decorate-sort-undecorate